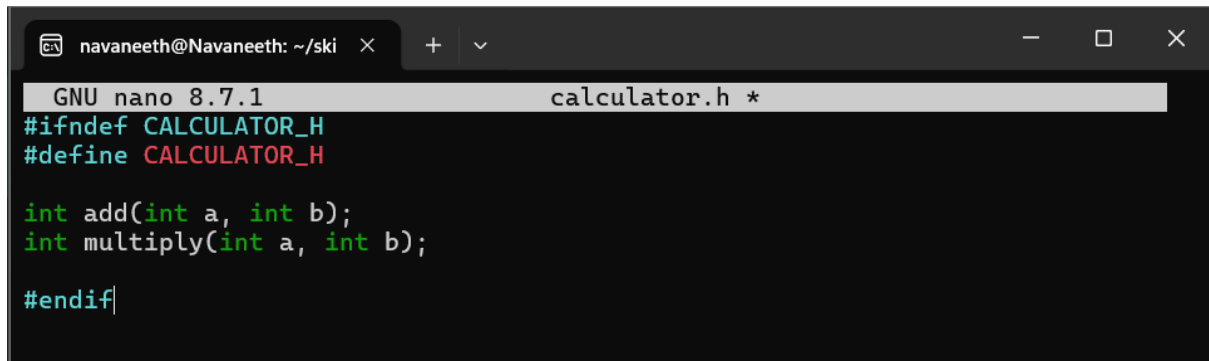


Skill Week 11

Aim

To create multiple C modules, connect them using a header file, compile and link the modules, validate the interface, and test the complete program execution.

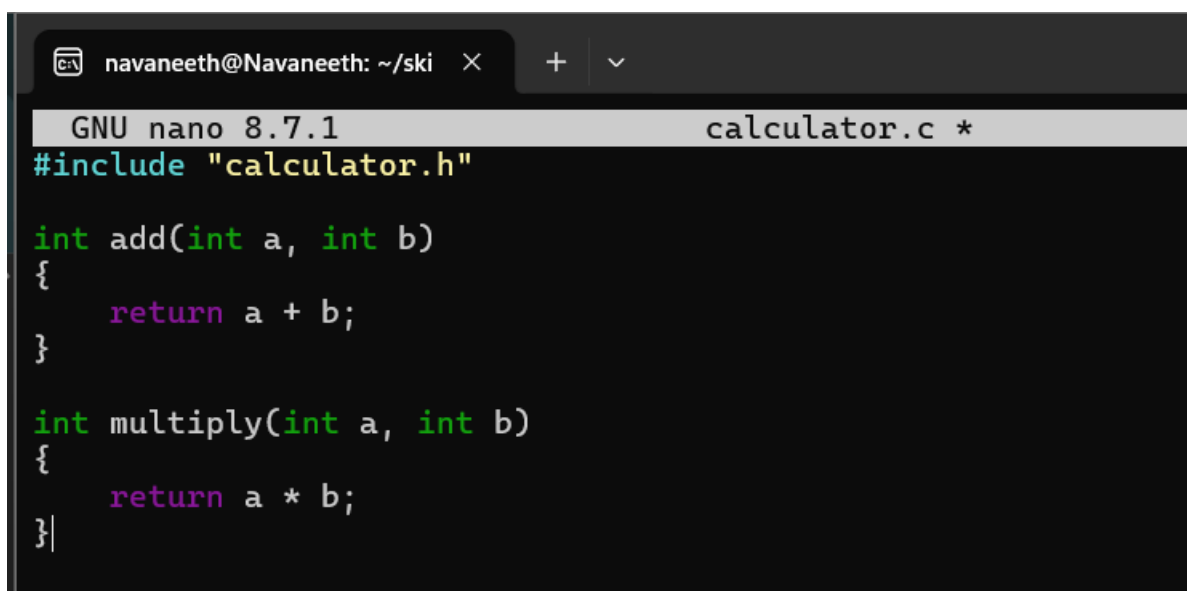
Program:



```
navaneeth@Navaneeth: ~/ski × + ▾
GNU nano 8.7.1 calculator.h *
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int multiply(int a, int b);

#endif
```



```
navaneeth@Navaneeth: ~/ski × + ▾
GNU nano 8.7.1 calculator.c *
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int multiply(int a, int b)
{
    return a * b;
}
```

```
navaneeth@Navaneeth: ~/ski × + v
GNU nano 8.7.1 main.c *
#include <stdio.h>
#include "calculator.h"

int main()
{
    int a = 10;
    int b = 5;

    printf("Addition = %d\n", add(a, b));
    printf("Multiplication = %d\n", multiply(a, b));

    return 0;
}
```

Output:

```
navaneeth@Navaneeth: ~/ski × + v - □ ×
navaneeth@Navaneeth:~$ mkdir -p skillweek11
navaneeth@Navaneeth:~$ cd skillweek11
navaneeth@Navaneeth:~/skillweek11$ nano calculator.h
navaneeth@Navaneeth:~/skillweek11$ nano calculator.c
navaneeth@Navaneeth:~/skillweek11$ nano main.c
navaneeth@Navaneeth:~/skillweek11$ gcc -c calculator.c
navaneeth@Navaneeth:~/skillweek11$ gcc -c main.c
navaneeth@Navaneeth:~/skillweek11$ gcc main.o calculator.o -o calculator
navaneeth@Navaneeth:~/skillweek11$ ./calculator
Addition = 15
Multiplication = 50
navaneeth@Navaneeth:~/skillweek11$ ls
calculator calculator.c calculator.h calculator.o main.c main.o
navaneeth@Navaneeth:~/skillweek11$ nano main.c
navaneeth@Navaneeth:~/skillweek11$ gcc -c main.c
main.c: In function 'main':
main.c:9:34: error: implicit declaration of function 'subtract' [-Wimplicit-
function-declaration]
   9 |         printf("Subtraction = %d\n", subtract(a, b));
     |                                     ^~~~~~
navaneeth@Navaneeth:~/skillweek11$ nano main.c
navaneeth@Navaneeth:~/skillweek11$ gcc -c main.c
navaneeth@Navaneeth:~/skillweek11$ gcc main.o calculator.o -o calculator
navaneeth@Navaneeth:~/skillweek11$ ./calculator
Addition = 15
Multiplication = 50
navaneeth@Navaneeth:~/skillweek11$ |
```

Observation

The header file provided the interface between main.c and calculator.c. Both modules were compiled separately and then linked together to produce one executable program. An invalid function interface generated a compilation warning/error.

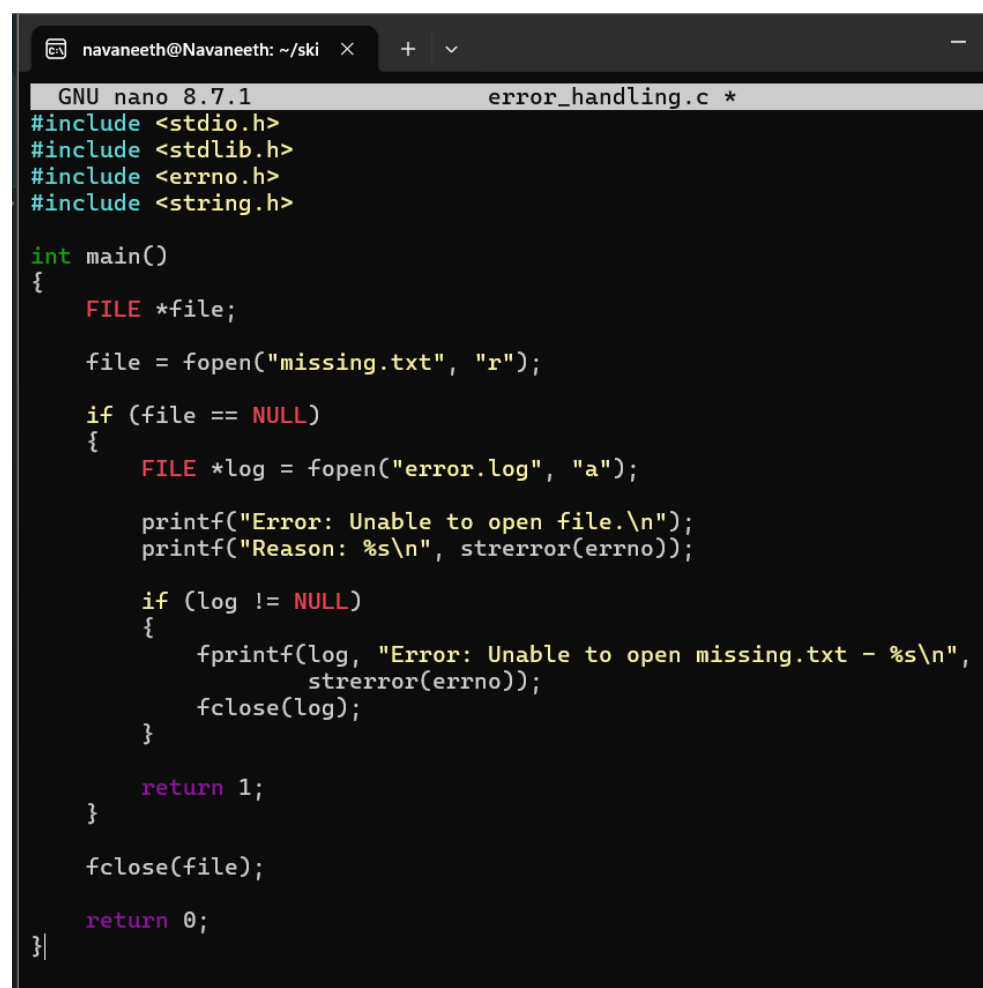
Result

Multiple C modules were successfully **connected, compiled, linked, tested, and validated for correct interface usage.**

Aim

To create a C program that detects common runtime errors, displays meaningful error messages, logs failures into a file, and handles invalid input safely.

Program:



```
GNU nano 8.7.1 error_handling.c *
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

int main()
{
    FILE *file;

    file = fopen("missing.txt", "r");

    if (file == NULL)
    {
        FILE *log = fopen("error.log", "a");

        printf("Error: Unable to open file.\n");
        printf("Reason: %s\n", strerror(errno));

        if (log != NULL)
        {
            fprintf(log, "Error: Unable to open missing.txt - %s\n",
                    strerror(errno));
            fclose(log);
        }

        return 1;
    }

    fclose(file);

    return 0;
}
```

Output:

```
navaneeth@Navaneeth:~/skillweek11$ nano error_handling.c
navaneeth@Navaneeth:~/skillweek11$ gcc error_handling.c -o error_handling
navaneeth@Navaneeth:~/skillweek11$ ./error_handling
Error: Unable to open file.
Reason: No such file or directory
navaneeth@Navaneeth:~/skillweek11$ cat error.log
Error: Unable to open missing.txt - No such file or directory
navaneeth@Navaneeth:~/skillweek11$ touch missing.txt
navaneeth@Navaneeth:~/skillweek11$ ./error_handling
navaneeth@Navaneeth:~/skillweek11$ ls
calculator      calculator.h  error.log      error_handling.c  main.o
calculator.c    calculator.o  error_handling  main.c            missing.txt
navaneeth@Navaneeth:~/skillweek11$ |
```

Observation

When missing.txt did not exist, the program detected the file-opening failure using `fopen()` and generated an error message using `strerror(errno)`. The failure was also recorded in error.log. After creating the file, the error condition was removed.

Result

Runtime error detection, error-message generation, failure logging, and basic error recovery were successfully implemented and tested.

Aim

To use **Valgrind** to detect memory leaks and memory-related errors in a C program, properly release dynamically allocated memory, and verify that the program has no memory leaks.

Program:

```
navaneeth@Navaneeth: ~/ski × + v
GNU nano 8.7.1 memory_test.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *arr;

    arr = malloc(5 * sizeof(int));

    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        arr[i] = i + 1;
    }

    printf("Array values: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");

    return 0;
}
```

```
navaneeth@Navaneeth:~/skillweek11$ nano memory_test.c
navaneeth@Navaneeth:~/skillweek11$ gcc -g memory_test.c -o memory_test
navaneeth@Navaneeth:~/skillweek11$ ./memory_test
Array values: 1 2 3 4 5
navaneeth@Navaneeth:~/skillweek11$ valgrind --leak-check=full ./memory_test
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
navaneeth@Navaneeth:~/skillweek11$ sudo apt update
[sudo: authenticate] Password:
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [526 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [674 kB]
```

```

navaneeth@Navaneeth:~/skillweek11$ nano memory_test.c
navaneeth@Navaneeth:~/skillweek11$ gcc -g memory_test.c -o memory_test
navaneeth@Navaneeth:~/skillweek11$ valgrind --leak-check=full --show-leak-kinds=all
./memory_test
==2087== Memcheck, a memory error detector
==2087== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2087== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2087== Command: ./memory_test
==2087==
Array values: 1 2 3 4 5
==2087==
==2087== HEAP SUMMARY:
==2087==   in use at exit: 0 bytes in 0 blocks
==2087==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==2087==
==2087== All heap blocks were freed -- no leaks are possible
==2087==
==2087== For lists of detected and suppressed errors, rerun with: -s
==2087== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
navaneeth@Navaneeth:~/skillweek11$ ./memory_test
Array values: 1 2 3 4 5
navaneeth@Navaneeth:~/skillweek11$ valgrind --leak-check=full --show-leak-kinds=all
./memory_test
==2107== Memcheck, a memory error detector
==2107== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2107== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2107== Command: ./memory_test
==2107==
Array values: 1 2 3 4 5
==2107==
==2107== HEAP SUMMARY:
==2107==   in use at exit: 0 bytes in 0 blocks
==2107==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==2107==
==2107== All heap blocks were freed -- no leaks are possible
==2107==
==2107== For lists of detected and suppressed errors, rerun with: -s
==2107== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
navaneeth@Navaneeth:~/skillweek11$ |

```

Observation

The memory allocated using malloc() was successfully released using free(). Valgrind reports no memory leaks.

Result

Memory allocation and cleanup were verified successfully using Valgrind.

Aim

To design test cases, create a test script, automate program execution, verify outputs, compare results, and generate a test report.

Program:

```
navaneeth@Navaneeth: ~/ski × + v
GNU nano 8.7.1 test.sh *
#!/bin/bash

echo "==== Automated Test Report ====="

./calculator > actual.txt

echo "Addition = 15" > expected.txt
echo "Multiplication = 50" >> expected.txt

if diff -q actual.txt expected.txt > /dev/null
then
    echo "Test 1: PASS"
    echo "Output verification: PASS"
else
    echo "Test 1: FAIL"
    echo "Output verification: FAIL"
fi

echo "====="

echo "Test 1: PASS" > test_report.txt
echo "Output verification completed." >> test_report.txt
```

Output:

```
navaneeth@Navaneeth:~/skillweek11$ nano test.sh
navaneeth@Navaneeth:~/skillweek11$ chmod +x test.sh
navaneeth@Navaneeth:~/skillweek11$ ./test.sh
==== Automated Test Report =====
Test 1: PASS
Output verification: PASS
=====
navaneeth@Navaneeth:~/skillweek11$ cat test_report.txt
Test 1: PASS
Output verification completed.
navaneeth@Navaneeth:~/skillweek11$ cat actual.txt
Addition = 15
Multiplication = 50
navaneeth@Navaneeth:~/skillweek11$ cat expected.txt
Addition = 15
Multiplication = 50
navaneeth@Navaneeth:~/skillweek11$ |
```

Observation

The test script automatically executed the program, compared the actual output with the expected output, and generated a test report.

Result

Automated testing, output comparison, and report generation were completed successfully.